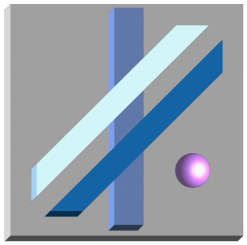

Fixed Point Solutions, LLC

Relend Network Stablecoin System Assessment

2025/01/26
Prepared by: Kurt Barry

1. Scope

The Relend Network Stablecoin System is a multi-domain stablecoin protocol that aims to provide a degree of isolation between stablecoin instances on different chains, while also employing a fractional reserve model to relend the backing assets, generating additional income for those that provide insurance to the protocol via a Morpho vault. It will be initially deployed on Ethereum mainnet, targeting Ethereum L2s for isolated stablecoin deployments.

The contents of the src/L1/ directory were reviewed at commit d60ed8168df966bbdc7473272ba08c6a371ebb33 with approximately 8 hours of effort by a single reviewer. Fixes were made and reviewed for all issues identified. The commit used for fix review was 884fb467f149b4f21ca89a2aedcad44c7331ae15.

Notable assumptions of the review included that all privileged roles were trusted, compliance with the draft EIP-7770 was out-of-scope, and that if the price of a Morpho collateral were ever changed to allow liquidations the protocol team would perform those liquidations atomically with the price change.

2. Limitations

No assessment can guarantee the absolute safety or security of a software-based system. Further, a system can become unsafe or insecure over time as it and/or its environment evolves. This assessment aimed to discover as many issues and make as many suggestions for improvement as possible within the specified timeframe. Undiscovered issues, even serious

ones, may remain. Issues may also exist in components and dependencies not included in the assessment scope.

3. Findings

Findings and recommendations are listed in this section, grouped into broad categories. It is up to the team behind the code to ultimately decide whether the items listed here qualify as issues that need to be fixed, and whether any suggested changes are worth adopting. When a response from the team regarding a finding is available, it is provided.

Findings are given a severity rating based on their likelihood of causing harm in practice and the potential magnitude of their negative impact. Severity is only a rough guideline as to the risk an issue presents, and all issues should be carefully evaluated.

Severity Level Determination		Impact		
		High	Medium	Low
Likelihood	High	**Critical**	**High**	**Medium**
	Medium	**High**	**Medium**	**Low**
	Low	**Medium**	**Low**	**Low**

Issues that do not present any quantifiable risk (as is common for issues in the Code Quality category) are given a severity of **Informational**.

Summary of Findings	
Severity	Count
Critical	0
High	0
Medium	0
Low	1
Informational	2

3.1 Safety and Correctness

Findings that could lead to harmful outcomes or violate the intentions of the system.

SC.1 Tokens With Certain Non-Standard `approve()` Functions Are Incompatible With the Protocol

Severity: Low

Code Location:
https://github.com/backstop-protocol/ERC-7770/blob/d60ed8168df966bbdc7473272ba08c6a371ebb33/src/L1/MorphoBank.sol#L59-L60

Description: Some tokens, notably USDT, have an `approve()` function that reverts if the existing approval is non-zero and also fails to return a boolean value. A core feature of the protocol is that the same underlying asset can be used to back multiple cross-chain wrapped tokens ("wtokens"). The `MorphoBank.listWToken()` function is used to register new wtokens and always casts the underlying token to `IERC20` and calls `approve()` directly, including on the `MORPHO` address which is immutable. This is incompatible with USDT-like tokens in two ways: first, the `IERC20` interface used will cause the compiler to insert a check for appropriate-length returndata. Second, even without that issue, the second attempt to call `approve()` would always fail due to the existing approval.

Recommendation: Handle such exceptional tokens gracefully, for example via OpenZeppelin's `SafeERC20.forceApprove()` function which deals with both issues.

Response: Fixed in PR#4.

FPS: Fix verified.

3.2 Usability and Incentives

Findings that could lead to suboptimal user experience, hinder integrations, or lead to undesirable behavioral outcomes.

U.1 Consider Indexing the `_wtoken` Parameter of `MorphoBank` Events

Severity: Informational

Code Location:
https://github.com/backstop-protocol/ERC-7770/blob/d60ed8168df966bbdc7473272ba08c6a371ebb33/src/L1/MorphoBank.sol#L23-L26

Description: It would be easier to filter for all events that apply to a specific wtoken if the `_wtoken` parameter used in `MorphoBank` events were indexed.

Response: Fixed in PR#4.

FPS: Fix verified.

3.3 Gas Optimizations

Findings that could reduce the gas costs of interacting with the protocol, potentially on an amortized or averaged basis.

Gas optimization was not a strong focus of this review. While optimizations are certainly possible, no particularly impactful ones were identified.

3.4 Code Quality

CQ.1 When Linking to External Code, Use Commit-Based Links

Severity: **Informational**

Code Location:
https://github.com/backstop-protocol/ERC-7770/blob/d60ed8168df966bbdc7473272ba08c6a371ebb33/src/L1/MorphoBank.sol#L50

Description: Links to code that use branches or other mutable aliases are prone to breakage or simply not pointing to the intended code any longer, making it harder to understand and verify the code in the future. Commit-based links are generally more reliable.

Response: Fixed in PR#4.

FPS: Fix verified.